

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ  
РОССИЙСКОЙ ФЕДЕРАЦИИ ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ  
БЮДЖЕТНОЕ  
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Институт математики и информационных систем  
Факультет автоматики и вычислительной техники  
Кафедра электронных вычислительных машин

Отчёт по лабораторной работе №1  
по дисциплине  
«Основы промпт-инжиниринга: обработка текста»  
«Вариант 4»

Выполнил студент гр. ИВТб-1303-06-00

\_\_\_\_\_/Гортоломей И.К./

Проверил доцент кафедры ЭВМ

\_\_\_\_\_/Колупаев А.В./

Киров

2026

## Цель работы:

Освоение фундаментальных и продвинутых методов разработки промптов, включая генерацию без примеров, обучение на примерах, цепочки рассуждений и ролевое моделирование. Научиться формализовать неструктурированные требования в алгоритмические последовательности запросов, а также проводить сравнительный анализ эффективности различных языковых моделей при решении прикладных задач обработки естественного языка.

## Используемые языковые модели:

«YandexGPT 5.1 Pro», «DeepSeek-V3»

## 1 задание: Генерация текста по заданной структуре

### 1.1 исходные данные

Тема: **NoSQL** базы данных

Тип документа: **Статья в блог**

Целевая аудитория: **Студенты IT**

Обязательные разделы:

**Типы БД, CAP-теорема, Примеры использования**

Тональность: **Неформальная**

### 1.2 базовый промпт

Ты Сеньёр-разработчик с 10-ти летним опытом работы.

Напиши {ТИП\_ДОКУМЕНТА} на тему {ТЕМА},  
обязательно включая разделы: {РАЗДЕЛЫ}.

Твоя целевая аудитория: {АУДИТОРИЯ},  
а тональность текста: {ТОНАЛЬНОСТЬ}.

Я выбрал роль Сеньёр-разработчика для ии так как обычно они с таким стажем уже много где работали и точно встречали разные виды баз данных.

### 1.3 журнал промпт-инжиниринга

Промпт v1:

Ты Сеньёр-разработчик с 10-ти летним опытом работы.  
Напиши Статья в блог на тему NoSQL базы данных,  
обязательно включая разделы: Типы БД, CAP-теорема,  
Примеры использования.  
Твоя целевая аудитория: Студенты IT,  
а тональность текста: Неформальная.

Результат (YandexGPT):

Ты Сеньёр-разработчик с 10-ти летним опытом работы. Напиши Статью в блог на тему NoSQL базы данных, обязательно включая разделы: Типы БД, CAP-теорема, Примеры использования. Твоя целевая аудитория: Студенты IT, а тональность текста: Неформальная.

## NoSQL: Когда SQL уже не катит 🦄

Привет, юные падаваны IT-мира! Сегодня поговорим о том, что делать, когда традиционные SQL-базы данных начинают вас подводить. Пристегните ремни, будет жарко!

### Что такое NoSQL? 📖

**NoSQL** — это не просто модное словечко, а целый класс баз данных, которые появились, когда интернет начал расти как на дрожжах. Когда миллионы пользователей и терабайты данных перестали влезать в привычные реляционные базы.

### Типы NoSQL баз данных 🇮🇹

Давайте разберёмся, какие бывают NoSQL базы:

- **Ключ-значение (Key-Value)**  
Самые простые и быстрые. Как огромный словарь, где по ключу достаётся значение. Amazon DynamoDB и Redis — ваши лучшие друзья в мире кэширования.
- **Документориентированные**  
MongoDB и CouchDB позволяют хранить документы в формате JSON. Идеально для социальных сетей и блогов.
- **Колоночные**  
Cassandra и HBase хранят данные по столбцам. Супер для аналитики и обработки больших данных.
- **Графовые**  
Neo4j и Amazon Neptune созданы для хранения связей между данными. Лучший выбор для социальных графов и рекомендательных систем.

### CAP-теорема: Почему нельзя всё и сразу 🎯

**CAP-теорема** гласит, что база данных может обладать только двумя из трёх свойств:

- **Consistency** (Согласованность) — все узлы видят одни и те же данные
- **Availability** (Доступность) — каждый запрос получает ответ
- **Partition tolerance** (Устойчивость к разделению) — система продолжает работать при сбоях сети

## CAP-теорема: Почему нельзя всё и сразу 🎯

**CAP-теорема** гласит, что база данных может обладать только двумя из трёх свойств:

- **Consistency** (Согласованность) — все узлы видят одни и те же данные
- **Availability** (Доступность) — каждый запрос получает ответ
- **Partition tolerance** (Устойчивость к разделению) — система продолжает работать при сбоях сети

Представьте, что это как с пиццей: можно выбрать только два топинга из трёх. В NoSQL обычно жертвуют согласованностью ради доступности и устойчивости.

## Где это реально используется? 🚀

- **Социальные сети**  
Facebook и Twitter используют NoSQL для хранения постов и связей между пользователями.
- **E-commerce**  
Amazon и eBay применяют NoSQL для хранения каталогов товаров и пользовательских данных.
- **Гейминг**  
ММО-игры используют графовые базы для хранения связей между игроками и их достижениями.
- **IoT**  
Устройства интернета вещей генерируют огромные потоки данных, которые отлично хранятся в NoSQL.

## Почему это круто? 🍌

- Масштабируемость без головной боли
- Высокая производительность
- Гибкая схема данных
- Отказоустойчивость

## Заключение 🎯

NoSQL — это не замена SQL, а инструмент для решения конкретных задач. Когда данные растут как на дрожжах, а связи между ними становятся всё сложнее, NoSQL приходит на помощь.

Теперь вы знаете, почему крупные компании переходят на NoSQL. И когда вы начнёте разрабатывать масштабные проекты, эти знания точно пригодятся!

А на этом всё, до новых встреч в мире баз данных! 🌈

Анализ ошибок: Ответ слишком длинный

Корректировка: попросил писать кратко.

Промпт v2:

Ты Сеньёр-разработчик с 10-ти летним опытом работы.

Напиши Статья в блог на тему NoSQL базы данных,

обязательно включая разделы: Типы БД, CAP-теорема,

Примеры использования.

Твоя целевая аудитория: Студенты IT,

а тональность текста: Неформальная.

Пиши максимально кратко, оставляя только самое важное.

## Результат (YandexGPT):

### NoSQL: когда SQL уже не катит

Привет, будущие гуру баз данных! Сегодня поговорим о NoSQL — крутой альтернативе классическим SQL-базам.

#### Что такое NoSQL?

NoSQL — это не просто "не SQL", а целый мир гибких баз данных, которые решают проблемы, с которыми SQL справиться не может.

#### Основные типы NoSQL

- **Ключ-значение** (Redis, DynamoDB) — простейшая структура, где данные хранятся как пара ключ-значение. Идеальны для кэширования.
- **Документориентированные** (MongoDB) — хранят данные в документах (обычно JSON). Отлично подходят для социальных сетей и e-commerce.
- **Колоночные** (Cassandra) — данные организованы по столбцам. Супер для аналитики и больших данных.
- **Графовые** (Neo4j) — хранят связи между данными. Лучшие друзья социальных сетей и рекомендательных систем.

#### CAP-теорема: почему нельзя всё и сразу

CAP-теорема гласит, что база данных может обладать только двумя из трёх свойств:

- **Согласованность** (Consistency) — все узлы содержат одинаковую информацию
- **Доступность** (Availability) — каждый запрос получает ответ
- **Устойчивость к разделению** (Partition tolerance) — система работает при сбоях сети

#### Где это реально используется

- **Социальные сети** — графовые БД хранят связи между пользователями
- **Маркетплейсы** — документориентированные БД для хранения разнородной информации о товарах
- **Игровые сервисы** — ключ-значение для быстрого доступа к данным пользователей
- **Аналитика** — колоночные БД для обработки огромных массивов данных

#### Почему стоит обратить внимание

NoSQL — это не замена SQL, а мощный инструмент для специфических задач. Если вы работаете с большими данными, сложными связями или нуждаетесь в высокой производительности — NoSQL может стать вашим лучшим другом.

P.S. Не бросайтесь сразу переписывать все проекты на NoSQL — сначала хорошенько подумайте, действительно ли вам это нужно!

Анализ ошибок: модель предоставила краткий вариант текста.

Корректировка: не требуется.

## Результат (DeepSeek):

🔗 Думал 7 секунд >

Привет, народ! 🧡

Забудьте на минутку про скучные лекции. Представьте, что ваша база данных — это не строгая библиотекарша в очках (как SQL), которая требует заполнить формуляр по правилам, прежде чем дать книгу. Сегодня поговорим про её крутых братьев и сестер — NoSQL. Это как огромный склад, куда можно скидывать всё что угодно и как угодно, лишь бы потом можно было найти.

Я как сеньёр, который перещупал кучь проектов, скажу так: **SQL — это надёжность и порядок**, **NoSQL — это скорость и масштаб**. Выбирай инструмент с умом. Погнали разбираться, что к чему.

---

### Типы БД: Какие они бывают?

NoSQL — это не одна конкретная база, а целая семейка. У каждого типа свой характер и своя задача.

#### 1. Ключ-Значение (Key-Value Stores)

Самые простые и быстрые. Работают как огромный словарь или хеш-таблица: ты даёшь ключ, получаешь значение.

- **Примеры:** Redis, Memcached.
- **В реальности:** Кэширование данных, сессии пользователей. Закинул объект пользователя по ключу `user:123` и достаешь за микросекунды.

#### 2. Документо-ориентированные (Document Stores)

Хранят данные в виде документов (JSON, BSON). Самый популярный тип для веб-разработки.

- **Примеры:** MongoDB, CouchDB.
- **В реальности:** Блоги, каталоги товаров. Вся информация о товаре (название, цена, характеристики, отзывы) лежит в одном документе. Никаких тебе JOIN-ов и танцев с бубном.

#### 3. Колоночные (Wide-Column Stores)

Данные организованы в колонки, а не в строки. Это позволяет очень быстро агрегировать данные по конкретному признаку.

- **Примеры:** Cassandra, HBase.
- **В реальности:** Аналитика, лог-файлы, интернет вещей (IoT). Нужно посчитать сумму продаж за вчера по всем пользователям? Колоночные БД сделают это мгновенно.

#### 4. Графовые (Graph Databases)

Здесь всё про связи. Данные хранятся в виде узлов (сущности) и ребер (связи между ними).

- **Примеры:** Neo4j.



- Примеры: Neo4j.
  - В реальности: Социальные сети (друзья друзей), рекомендательные системы, антифрод. Идеально, если связи в данных важнее самих данных.
- 

## САР-теорема: Почему нельзя быть идеальным

Это краеугольный камень мира NoSQL. Теорема говорит, что в распределенной системе (которая работает на нескольких серверах) можно иметь только два из трех свойств одновременно:

- **C (Consistency) — Согласованность:** После записи данных все видят одну и ту же версию.
- **A (Availability) — Доступность:** Любой запрос всегда получает ответ (даже если данные старые).
- **P (Partition Tolerance) — Устойчивость к разделению:** Система продолжает работать, если связь между серверами потерялась.

Выбирать тебе:

- **CP (Согласованность + Устойчивость):** Жертвуем доступностью ради актуальных данных. Если сервера потеряют связь друг с другом, система может на время остановиться, чтобы не плодить противоречия. *HBase, MongoDB (по дефолту).*
- **AP (Доступность + Устойчивость):** Жертвуем согласованностью. Система будет отвечать всегда, но в моменты разрыва связи данные на разных серверах могут отличаться. Потом они синхронизируются (это называется "согласованность в конечном счете"). *Cassandra, CouchDB.*
- **CA (Согласованность + Доступность):** На практике в распределенных системах не существует. Это про обычные монолитные SQL-базы на одном сервере.

Коротко: Если у тебя банк — бери CP (деньги считать надо точно). Если соцсеть — бери AP (лучше показать старый пост, чем вообще не открыть приложение).

---

## Примеры использования: Где и что юзать

Хватит теории, перейдем к хардкору. Где эти базы реально рулят?

### 1. MongoDB (Документная) для интернет-магазина.

- **Ситуация:** У тебя товары. У каждого товара куча характеристик, которые могут меняться (у ноутбука — процессор, у футболки — размер).
- **Решение:** В SQL тебе пришлось бы городить таблицы `товар`, `характеристики`, `значения_характеристик` и джойнить их. В MongoDB ты просто хранишь JSON с характеристиками внутри документа товара. Всё просто и быстро.

### 2. Redis (Ключ-значение) для ленты активности.

## 2. Redis (Ключ-значение) для ленты активности.

- **Ситуация:** Тебе нужно показывать пользователю "10 последних действий друзей".
- **Решение:** Хранишь ID этих действий в Redis-списке. Добавил новый экшен в начало списка — обрезал хвост, если там больше 10 элементов. Достал — отдал клиенту. Скорость — космос.

## 3. Cassandra (Колоночная) для логов сервера.

- **Ситуация:** Падают миллионы записей логов каждую секунду. Нужно сохранить всё и уметь быстро выбрать по временным меткам.
- **Решение:** Cassandra проектировалась для этого. Она умеет "рассыпать" данные по куче серверов и не падает при нагрузках. Пишем пачками, читаем по времени.

## 4. Neo4j (Графовая) для рекомендаций друзей.

- **Ситуация:** Нужно найти друзей друзей моего друга, которые еще не являются моими друзьями, но живут в моем городе.
- **Решение:** В SQL для такого запроса понадобится 5-6 JOIN-ов и сложная логика. В графовой БД ты просто говоришь: "Найди путь длиной в 3 шага от меня, исключи моих прямых друзей и отфильтруй по городу". Делается одним запросом.

## Итог

NoSQL — это не замена SQL, а инструмент для конкретных задач. Не пытайся всё засунуть в MongoDB, как и не пытайся любую аналитику считать в MySQL.

- Нужна четкая структура и транзакции? → **SQL**.
- Много данных и они постоянно меняются? → **Документные БД**.
- Надо быстро гонять кэш? → **Redis**.
- Достал JOIN-ы в соцсетях? → **Графовые**.

Выбирай с умом, и код будет в кайф! 🚀

Анализ ошибок: модель не следует инструкции.

Корректировка: не требуется, рекомендуется использовать другую модель.

## 1.4 Сравнение моделей:

| Критерий                | YandexGPT | DeepSeek |
|-------------------------|-----------|----------|
| Следование инструкции   | 5         | 3        |
| Отсутствие галлюцинаций | 5         | 5        |
| Форматирование          | 5         | 5        |
| Длина текста            | 5         | 3        |
| Источники информации    | 3         | 5        |
| <b>Итог</b>             | 4.6       | 4.2      |

Модель DeepSeek лучше ищет информацию, но плохо сжимает ответ. А модель YandexGPT быстро но плохо ищет информацию, зато хорошо работает с файлами, текстом и фото.

## 1.5 Финальный промпт:

Ты Сеньёр-разработчик с 10-ти летним опытом работы.  
Напиши {ТИП\_ДОКУМЕНТА} на тему {ТЕМА},  
обязательно включая разделы: {РАЗДЕЛЫ}.  
Твоя целевая аудитория: {АУДИТОРИЯ},  
а тональность текста: {ТОНАЛЬНОСТЬ}.  
Пиши максимально кратко, оставляя только самое важное.

## 1.6 Вывод по заданию:

В данном задании лучше всего подходит техника «Ролевое моделирование», так как даёт понять меделе какой специалист для составления документа необходим.

## 2 задание: Ререйтинг и адаптация стиля

### 2.1 исходные данные

Тема источника: **Новая функция**

Целевая аудитория: **Пользователи**

Требуемый стиль: **Простой**

Ограничение: **Твит (280 зн.)**

Фокус: **Удобство**

#### **Сгенерированный текст:**

Представляем вашему вниманию релиз протокола взаимодействия с вычислительным ядром платформы. В текущий билд интегрирован модуль асинхронной обработки векторов состояний, который инициируется через вызов API-эндпоинта `/api/v3/process` с передачей массива байткодов.

Ключевое улучшение заключается в имплементации механизма предиктивной выборки данных из L3-кэша, что позволило снизить задержки при исполнении тяжелых запросов. Внедренный алгоритм сжатия на лету (on-the-fly compression) на основе дельта-кодирования теперь автоматически применяется к полезной нагрузке (payload) для оптимизации пропускной способности канала. Логика обработки ошибок была расширена за счет добавления кодов состояния 429 Too Many Requests и 507 Insufficient Storage.

Для обеспечения идемпотентности транзакций в заголовках запросов теперь обязательно требуется передавать UUID версии 4 в поле X-Request-ID. Система гарантирует консистентность данных в соответствии с требованиями ACID благодаря использованию двухфазного коммита (2PC) при взаимодействии с распределенным хранилищем. Для обратной совместимости со старыми клиентами оставлен "запасной" (fallback) эндпоинт, однако его поддержка будет прекращена в следующем мажорном релизе. Обратите внимание на обновленную схему аутентификации: теперь обязательна передача JWT-токена в каждом запросе, а не только при установлении сессии.

## 2.2 базовый промпт

Перепиши это для {АУДИТОРИЯ}: {ТЕКСТ}

предполагаю, что такого простого запроса должно хватить.

## 2.3 журнал промпт-инжиниринга

Промпт v1:

Перепиши это для Пользователи: ...

Результат (YandexGPT):

## **Обновление протокола взаимодействия с вычислительным ядром**

### **Основные изменения релиза**

Представлен новый выпуск протокола взаимодействия с вычислительным ядром платформы. В текущей версии реализован **модуль асинхронной обработки** векторов состояний, доступный через API-эндпоинт `/api/v3/process`. Модуль принимает на вход массив байт-кодов для обработки.

### **Ключевые улучшения производительности**

В новой версии реализованы следующие оптимизации:

- Внедрен **механизм предиктивной выборки** данных из L3-кэша, что существенно снизило задержки при выполнении ресурсоемких запросов
- Добавлен **алгоритм динамического сжатия** (on-the-fly compression) на основе дельта-кодирования для автоматической оптимизации полезной нагрузки

### **Обновления системы обработки**

Внесены изменения в обработку ошибок:

- Добавлены новые коды состояния: **429 Too Many Requests** и **507 Insufficient Storage**
- Введена обязательная передача **UUID версии 4** в заголовке **X-Request-ID** для обеспечения идемпотентности транзакций

### **Гарантии целостности данных**

Система обеспечивает **консистентность данных** в соответствии с принципами ACID за счет использования **двухфазного коммита** при работе с распределенным хранилищем.

### **Обратная совместимость и безопасность**

- Сохраняется поддержка устаревшего **fallback-эндпоинта** (будет удален в следующем мажорном релизе)
- Обновлена схема аутентификации: теперь требуется передача **JWT-токена** в каждом запросе, а не только при инициализации сессии

Анализ ошибок: модель сгенерировала всё ещё технический документ.

Корректировка: дополнительная настройка запроса.

Промпт v2:

Ты опытный айтишник.

Тебя слушают: Пользователи.

Перескажи текст,

чтобы он был: Простой, фокусируясь на: Удобство.

Ограничение: Твит (280 зн.).

Текст: ...

Результат (YandexGPT):

 Новый релиз API!

Что улучшили:

- Быстрее обрабатываем запросы
- Автоматически сжимаем данные
- Добавили защиту от перегрузок

Важно:

- Нужен JWT-токен в каждом запросе
- Добавьте UUID в заголовок X-Request-ID
- Старый endpoint скоро уберут 

Анализ ошибок: текст стал сильно короче, осталась только суть.

Корректировка: не требуется.

Результат (DeepSeek):

🧠 Думал 9 секунд >

Всем привет! Вышло обновление платформы. Главное для удобства: тяжелые запросы теперь летают быстрее, трафика уходит меньше, а об ошибках (например, при перегрузке) система сообщает понятнее.

Важный нюанс: скоро перестанет работать старый способ подключения, и теперь в каждом запросе обязательно нужен токен. Учтите это, чтобы ваш сервис не лёг.

Анализ ошибок: модель не использовала термины по типу UUID или JWT-токен  
Корректировка: не требуется.

## 2.4 Сравнение моделей:

| Критерий                    | YandexGPT | DeepSeek |
|-----------------------------|-----------|----------|
| Структурная валидность      | 4         | 5        |
| Стилистическое соответствие | 5         | 5        |
| Фактическая точность        | 3         | 5        |
| Итог                        | 4         | 5        |

Модель DeepSeek лучше соответствует заданному стилю.

## 2.5 Финальный промпт:

Ты опытный айтишник.

Тебя слушают: {АУДИТОРИЯ}.

Перескажи текст,

чтобы он был: {СТИЛЬ}, фокусируясь на: {ФОКУС}.

Ограничение: {ОГРАНИЧЕНИЕ}.

Текст: {ТЕКСТ}



## 2.6 Вывод по заданию:

В данном задании лучше всего подходит техника «Ролевое моделирование» так как для пересказа лучше всего подходит конкретный специалист.

## 3 задание: Перевод и локализация с сохранением технической спецификации

### 3.1 исходные данные

Исходный текст (фрагмент): "Para mantener la velocidad del equipo de desarrollo, es fundamental cumplir con los tiempos establecidos en la planificación. El deadline para el sprint actual es mañana al mediodía, y todavía hay tareas pendientes críticas. Por favor, revisen el backlog y actualicen el estado de sus tickets lo antes posible. No podemos cerrar el ciclo si las historias de usuario no están completas y probadas. Si hay algún bloqueo técnico, levántenlo en la reunión diaria para resolverlo antes del cierre."

Направление: Spa -> Rus

Запрет перевода: Agile термины (deadline, sprint, backlog)

### 3.2 базовый промпт

Переведи в направлении {НАПРАВЛЕНИЕ} этот текст: {ТЕКСТ}

простой запрос для перевода текста без уточнений.

### 3.3 журнал промпт-инжиниринга

Промпт v1:

Переведи в направлении Spa -> Rus этот текст: ...

Результат (YandexGPT):

Переведи в направлении Spa -> Rus этот текст: "Para mantener la velocidad del equipo de desarrollo, es fundamental cumplir con los tiempos establecidos en la planificación. El deadline para el sprint actual es mañana al mediodía, y todavía hay tareas pendientes críticas. Por favor, revisen el backlog y actualicen el estado de sus tickets lo antes posible. No podemos cerrar el ciclo si las historias de usuario no están completas y probadas. Si hay algún bloqueo técnico, levántenlo en la reunión diaria para resolverlo antes del cierre."

Для поддержания скорости работы команды разработчиков крайне важно соблюдать установленные в плане сроки. **Дедлайн** текущего спринта — завтра в полдень, а критические задачи всё ещё остаются невыполненными.

Пожалуйста, проверьте **бэклог** и как можно скорее обновите статус ваших задач. Мы не сможем закрыть цикл, если пользовательские истории не будут завершены и протестированы.

Если у вас возникли какие-либо технические препятствия (**блокировки**), поднимите их на ежедневной встрече, чтобы решить до завершения спринта.

📄 👍 🗨 ...

Анализ ошибок: модель полностью перевела текст даже с терминами, но выделила термины жирным.

Корректировка: добавить уточнение о запрете перевода терминов.

Промпт v2:

Переведи в направлении Spa -> Rus, но не переведи Agile термины (deadline, sprint, backlog). текст: ...

Результат (YandexGPT):

Переведи в направлении Spa -> Rus, но не переведи Agile термины (deadline, sprint, backlog). текст: "Para mantener la velocidad del equipo de desarrollo, es fundamental cumplir con los tiempos establecidos en la planificación. El deadline para el sprint actual es mañana al mediodía, y todavía hay tareas pendientes críticas. Por favor, revisen el backlog y actualicen el estado de sus tickets lo antes posible. No podemos cerrar el ciclo si las historias de usuario no están completas y probadas. Si hay algún bloqueo técnico, levántenlo en la reunión diaria para resolverlo antes del cierre."

Для поддержания скорости работы команды разработки крайне важно соблюдать установленные в планировании сроки. **Deadline** для текущего **sprint** — завтра в полдень, а критические задачи всё ещё остаются невыполненными.

Пожалуйста, проверьте **backlog** и обновите статус своих тикетов как можно скорее. Мы не сможем закрыть цикл, если пользовательские истории не будут завершены и протестированы.

Если есть какие-либо технические блокировки, поднимите их на ежедневной встрече, чтобы решить их до закрытия.

📄 👍 🗨 ...

Анализ ошибок: модель перевела текст, не переводя термины.

Корректировка: не требуется.

Результат (DeepSeek):

Переведи в направлении Spa -> Rus, но не переведи Agile термины (deadline, sprint, backlog). текст: "Para mantener la velocidad del equipo de desarrollo, es fundamental cumplir con los tiempos establecidos en la planificación. El deadline para el sprint actual es mañana al mediodía, y todavía hay tareas pendientes críticas. Por favor, revisen el backlog y actualicen el estado de sus tickets lo antes posible. No podemos cerrar el ciclo si las historias de usuario no están completas y probadas. Si hay algún bloqueo técnico, levántenlo en la reunión diaria para resolverlo antes del cierre."



 Думал 7 секунд >

Для поддержания скорости команды разработки крайне важно соблюдать сроки, заложенные в планировании. *Deadline* текущего *sprint* — завтра в полдень, а критических задач в работе все еще много. Пожалуйста, проверьте *backlog* и обновите статус своих тикетов как можно скорее. Мы не сможем закрыть цикл, если пользовательские истории не будут завершены и протестированы. Если есть какие-то технические блокеры, поднимите их на ежедневной встрече, чтобы мы успели решить их до закрытия.



Анализ ошибок: эта модель перевела текст аналогично.

Корректировка: не требуется.

Результат (DeepSeek, на варианте 11):

Переведи в направлении Eng -> Rus, но не переведи Команды Unix (chmod 777), пути (public\_html). текст: "Web server security heavily relies on proper file permission settings to prevent unauthorized access. Administrators must make sure that chmod 777 is NOT used on the public\_html directory, as it grants full read, write, and execute permissions to everyone. Such a configuration is a critical vulnerability that allows attackers to upload malicious scripts. Instead, use more restrictive permissions like 755 for directories and 644 for files. Regularly audit the file system to identify and fix insecure permission settings."



🔍 Думал 5 секунд >

Безопасность веб-сервера во многом зависит от правильной настройки прав доступа к файлам, что позволяет предотвратить несанкционированный доступ. Администраторы должны убедиться, что `chmod 777` НЕ используется для каталога `public_html`, так как это даёт полные права на чтение, запись и выполнение всем пользователям. Такая конфигурация является критической уязвимостью, позволяющей злоумышленникам загружать вредоносные скрипты. Вместо этого используйте более ограничительные права, такие как `755` для каталогов и `644` для файлов. Регулярно проверяйте файловую систему, чтобы выявлять и исправлять небезопасные настройки прав.



Анализ ошибок: данный промпт хорошо работает и для других вариантов.  
Корректировка: не требуется.

### 3.4 Сравнение моделей:

| Критерий                      | YandexGPT | DeepSeek |
|-------------------------------|-----------|----------|
| Терминологическая целостность | 5         | 5        |
| Сохранение смысла             | 4         | 5        |
| <b>Итог</b>                   | 4.5       | 5        |

Модель DeepSeek справилась лучше, так как больше вникает в контекст и имеет функцию "размышления" что позволяет более точно перевести текст, зато YandexGPT быстрее обрабатывает запросы но имеет жёсткий лимит на кол-во токенов вывода.

### 3.5 Финальный промпт:

Переведи в направлении {НАПРАВЛЕНИЕ}, но не переводи {ЗАПРЕТ\_ПЕРЕВОДА}. текст: ТЕКСТ}

### 3.6 Вывод по заданию:

В данном задании лучше всего подходит техника «Цепочка рассуждений» так как модель лучше вникает в контекст.

## 4 задание: Суммаризация (Сокращение текста)

### 4.1 исходные данные

Тема исходного текста: **Компьютерное зрение**

Формат вывода: **Таблица сравнения**

Степень сжатия: **Средняя**

Фокус внимания: **Скорость и точность**

Ссылка на статью с текстом:

**<https://habr.com/ru/articles/946080/> (1500+ слов)**

Фрагмент текста:

Инженерные чертежи содержат десятки типов размеров и допусков: линейные и угловые, радиальные и диаметральные, справочные и базовые, а также геометрические характеристики вроде плоскостности или перпендикулярности. Для инженера это привычный язык, но для автоматической обработки такие документы представляют серьёзную задачу.

С появлением мультимодальных LLM возник вопрос: способны ли они заменить классические OCR-решения и специализированные пайплайны постобработки? Насколько хорошо современные модели справляются с извлечением размеров и допусков из коробки, без дообучения? Какие модели дают максимальную точность, какие быстрее, а какие дешевле? И можно ли комбинировать разные подходы, чтобы повысить результат?

Чтобы ответить на эти вопросы, мы протестировали шесть актуальных vision-LLM на одном наборе реальных механических чертежей и сравнили их точность, скорость и стоимость обработки.

<< ... >>

## 4.2 базовый промпт

Сделай краткое содержание этого текста: {ТЕКСТ}

Самый простой и понятный запрос для сжатия текста.

## 4.3 журнал промпт-инжиниринга

Промпт v1:

Сделай краткое содержание этого текста: ...



Результат (YandexGPT):

## **Краткое содержание исследования по обработке инженерных чертежей LLM-моделями**

### **Предмет исследования**

Тестирование моделей на способность автоматического извлечения размеров и допусков из инженерных чертежей без дополнительного обучения.

### **Методология**

- Использовано **10 реальных чертежей** с разным количеством аннотаций (17-58 размеров)
- Тестировались 6 мультимодальных LLM: Gemini 2.5 Pro/Flash, Claude Opus 4, ChatGPT o4 mini/o3, Qwen VL Plus
- Оценивались точность, скорость и стоимость обработки

### **Основные результаты**

Лидеры тестирования:

- Gemini 2.5 Pro — 80% точности
- Gemini 2.5 Flash — 77% точности

Отстающие модели:

- Qwen VL Plus — всего 8% точности
- ChatGPT o3 — 20% точности

### **Ключевые наблюдения**

- Модели стабильно ошибались при обработке геометрических допусков
- Сложность чертежа не всегда влияла на точность распознавания
- Оптимальным по соотношению цена/качество оказался Gemini Flash

### **Рекомендации**

- Использовать **ансамбль моделей** для повышения точности
- Применять **итеративную обработку** одного чертежа
- Проводить **человеческую верификацию** критически важных данных

### **Выводы**

- Автоматическое распознавание чертежей возможно, но требует дополнительной обработки
- Для промышленного применения необходима интеграция в специальный пайплайн
- Перспективность технологии подтверждается растущим качеством моделей
- В текущих условиях рекомендуется использовать Gemini Flash как оптимальное решение

📄 👍 🗨️ ...

Анализ ошибок: модель предоставила ответ в другом формате.

Корректировка: добавить уточнение в запрос.

Промпт v2:

Сделай краткое содержание текста со степенью сжатия: Средняя.

Формат вывода содержимого строго только: Таблица сравнения.

Фокус внимания в содержимом на: Скорость и точность.

текст: ...

## Результат (YandexGPT):

| Модель                      | Средняя<br>точность<br>извлечения<br>(%) | Время<br>обработки<br>(сек/стр) | Стоимость<br>за 1000 стр<br>(USD) | Ключевые особенности                                                                                  |
|-----------------------------|------------------------------------------|---------------------------------|-----------------------------------|-------------------------------------------------------------------------------------------------------|
| <b>Gemini<br/>2.5 Pro</b>   | 79,96                                    | 91,4                            | 130,4                             | <b>Лидер</b> по точности,<br>стабильно работает со<br>сложными чертежами,<br>хорошо извлекает допуски |
| <b>Gemini<br/>2.5 Flash</b> | 77,34                                    | 77,5                            | 30,5                              | <b>Оптимальный баланс</b><br>точности и цены, высокая<br>стабильность                                 |
| <b>ChatGPT<br/>o4 mini</b>  | 39,59                                    | 41,75                           | 24,9                              | <b>Средняя эффективность</b> ,<br>лучше работает с<br>"чистыми" чертежами                             |
| <b>Claude<br/>Opus 4</b>    | 40,49                                    | 64,8                            | 312                               | <b>Высокая стоимость</b> ,<br>нестабильная работа при<br>высокой плотности<br>аннотаций               |
| <b>ChatGPT<br/>o3</b>       | 20,38                                    | 163                             | 239,2                             | <b>Низкая эффективность</b> ,<br>медленная обработка                                                  |
| <b>Qwen VL<br/>Plus</b>     | 7,23                                     | 22                              | 1,59                              | <b>Самая быстрая</b> , но крайне<br>низкая точность                                                   |

### Общие выводы по производительности:

- Модели показали разную эффективность в зависимости от сложности чертежа
- Все модели испытывали трудности с геометрическими допусками
- Оптимальным выбором для промышленного применения является **Gemini Flash**
- Для максимальной точности рекомендуется использовать комбинацию моделей

### Важные наблюдения:

- Сложность чертежа не всегда коррелирует с точностью извлечения
- Стандартизированность оформления важнее количества размеров
- Даже лучшие модели требуют последующей верификации данных

Анализ ошибок: модель отклонилась от заданной структуры.

Корректировка: использовать другую модель.

Результат (DeepSeek):

**Сравнение моделей по точности и скорости (средние значения)**

| Модель           | Точность | Время (сек/стр)       | Стоимость (\$/1000 стр.) | Ключевые особенности                                                                        |
|------------------|----------|-----------------------|--------------------------|---------------------------------------------------------------------------------------------|
| Gemini 2.5 Pro   | ~80%     | 91,4                  | 130,4                    | Высшая точность, стабильна на сложных чертежах, иногда объединяет одинаковые размеры.       |
| Gemini 2.5 Flash | ~77%     | 77,5                  | 30,5                     | Оптимальный баланс цены, скорости и точности, на одном чертеже показал результат лучше Pro. |
| Claude Opus 4    | ~40%     | 64,8                  | 312                      | Неплох на простых чертежах, но дорогой и не лидирует по точности.                           |
| ChatGPT o4 mini  | ~39,6%   | 41,75 (самый быстрый) | 24,9 (дешёвый)           | Хорош для линейных размеров, но путает допуски и теряет точность на сложных схемах.         |
| ChatGPT o3       | ~20%     | 163 (самый медленный) | 239,2                    | Низкая точность и высокая цена, часто не находит идентификаторы размеров.                   |
| Qwen VL Plus     | ~7%      | 22 (самый быстрый)    | 1,59 (самый дешёвый)     | Очень быстро и дёшево, но точность крайне низкая, не делит чертёж на проекции.              |

Анализ ошибок: модель предоставила таблицу сравнения.

Корректировка: не требуется.

#### 4.4 Сравнение моделей:

| Критерий                 | YandexGPT | DeepSeek |
|--------------------------|-----------|----------|
| Информационная плотность | 5         | 5        |
| Отсутствие искажений     | 5         | 5        |
| Следование инструкции    | 3         | 5        |
| <b>Итог</b>              | 4.33      | 5        |

Модель DeepSeek снова выдала ответ лучше, так как имеет функцию "подумать" что позволяет сгенерировать более валидный результат, а модель YandexGPT не следует инструкции и добавляет лишние разделы.

#### 4.5 Финальный промпт:

Сделай краткое содержание текста со степенью сжатия: {СТЕПЕНЬ}.

Формат вывода содержимого строго только: {ФОРМАТ}.

Фокус внимания в содержимом на: {ФОКУС}.

текст: {ТЕКСТ}

#### 4.6 Вывод по заданию:

В данном задании рекомендуется использовать технику «Уплотнение смысла», но мне показадлсь что для сжатия всей статьи в таблицу сравнения достаточно и обычного запроса.

## 5 задание: Классификация текстов

### 5.1 исходные данные

Входные данные: **Заголовки новостей**

Категории (Классы): **[Politics,Sports,Tech,Finance]**

Формат вывода: **JSON:headline,topic**